



SOICT

PROJECT 3 REPORT

Laws & General Documents Multi Agent-Based System

Course: IT3940E

Supervisor: Assoc. Prof. Pham Van Hai

Authors:

Trieu Kinh Quoc

Student ID:

20225524

Hanoi, December 2025

Acknowledgements:

This work is supported and supervised by Assoc. Prof. Pham Van Hai under the course of *Project 3 - IT3940E*. Also, our research and project are partially supported by other students at School of Information and Communication Technology, Hanoi University of Science and Technology.

Table of Contents

1 Introduction	5
2 Proposed System	5
2.1 System Design	5
2.2 Used Framework	7
3 Methodology	8
3.1 Different types of Router	8
3.1.1 Key-word Matching Based	8
3.1.2 Prompt-based Router	8
3.1.3 Classifier-based Router	8
3.1.4 Embedding Similarities Router	9
3.1.5 Evaluation Result	9
3.2 RAG (Vector Database)	10
3.2.1 Reciprocal Ranking	10
3.2.2 Index Implementation	10
3.2.2.1 Different Kinds of Indexing Strategies	11
3.2.2.2 Evaluation Results	11
3.3 Multi Agent System Simulation with LangGraph	13
4 Agent Details	13
4.1 Router Agent	13
4.2 Generate Sub-queries Agent	14
4.3 Retrieve Laws Agent	14
4.4 Laws Answer Agent	14
4.5 Extract Documents Agent	14
4.6 Documents Answer Agent	15
4.7 Verifier Agent	15
4.8 Reasoning Agent	15
5 Evaluation and Experimental Testing	15
5.1 Agent Performance	15
5.1.1 Router Agent	15
5.1.2 Generate Sub-queries Agent	16
5.1.3 Retrieve Laws Agent	16
5.1.4 Laws Answer Agent	18
5.1.5 Extract Documents Agent	19
5.1.6 Documents Answer Agent	19
5.2 Agent Processing Time	20
5.2.1 Experimental Environment	20
5.2.2 Experimental Result	20
6 Link to Dataset and Source Code:	21

6.1 Dataset	21
6.2 Source Code	21
7 Discussion	21
7.1 Conclusion	21
7.2 Future Work	21

Abstract

Multi-Agent architectures and Retrieval-Augmented Generation (RAG) are gaining interest in the provision of specialized domain consultation and document analysis. An essential requirement in such systems is the accurate disambiguation of user intent and the strict adherence to factual grounding in generated responses. In this report, i present a robust Multi-Agent System for Vietnamese legal answer and general document question-answering, developed using the LangGraph framework. The proposed system is designed to provide systematic support for users by combining specialized agents based on query context.

Keywords: Multi-Agent System, Retrieval-Augmented Generation (RAG), Legal Chatbot, LangGraph, DeepEval, Vietnamese Law, LLM-as-a-judge.

1 Introduction

The rapid development of legal and the increasing volume of informative documents stored in unstructured formats, such as PDF, have necessitated the development of more sophisticated information retrieval and reasoning systems. While Large Language Models have demonstrated significant potential in natural language understanding, the legal domain imposes rigorous demands for precision, traceability, and the ability to answer the complicated tasks that often exceed the functional limitations of single-agent architectures. To address these challenges, the project proposes a Multi-Agent System designed to orchestrate a collaborative workflow between specialized autonomous agents. By decomposing the legal question-answering process into distinct modules—such as PDF documents parsing, semantic retrieval, and answer validation — this framework utilizes a “divide-and-conquer” paradigm to ensure the accuracy of generated answer. Through iterative reasoning and coordination, the system aims to bridge the gap between raw, unstructured legal data in order to provide a robust and scalable solution for modern legal analysis and information synthesis.

This report is divided into five main sections: the introduction of the study; the related research for this problem; analysis of algorithms that have been used to solve problems; the evaluation results and the conclusion about this work.

2 Proposed System

2.1 System Design

The design of proposed multi-agent system can be seen bellow:

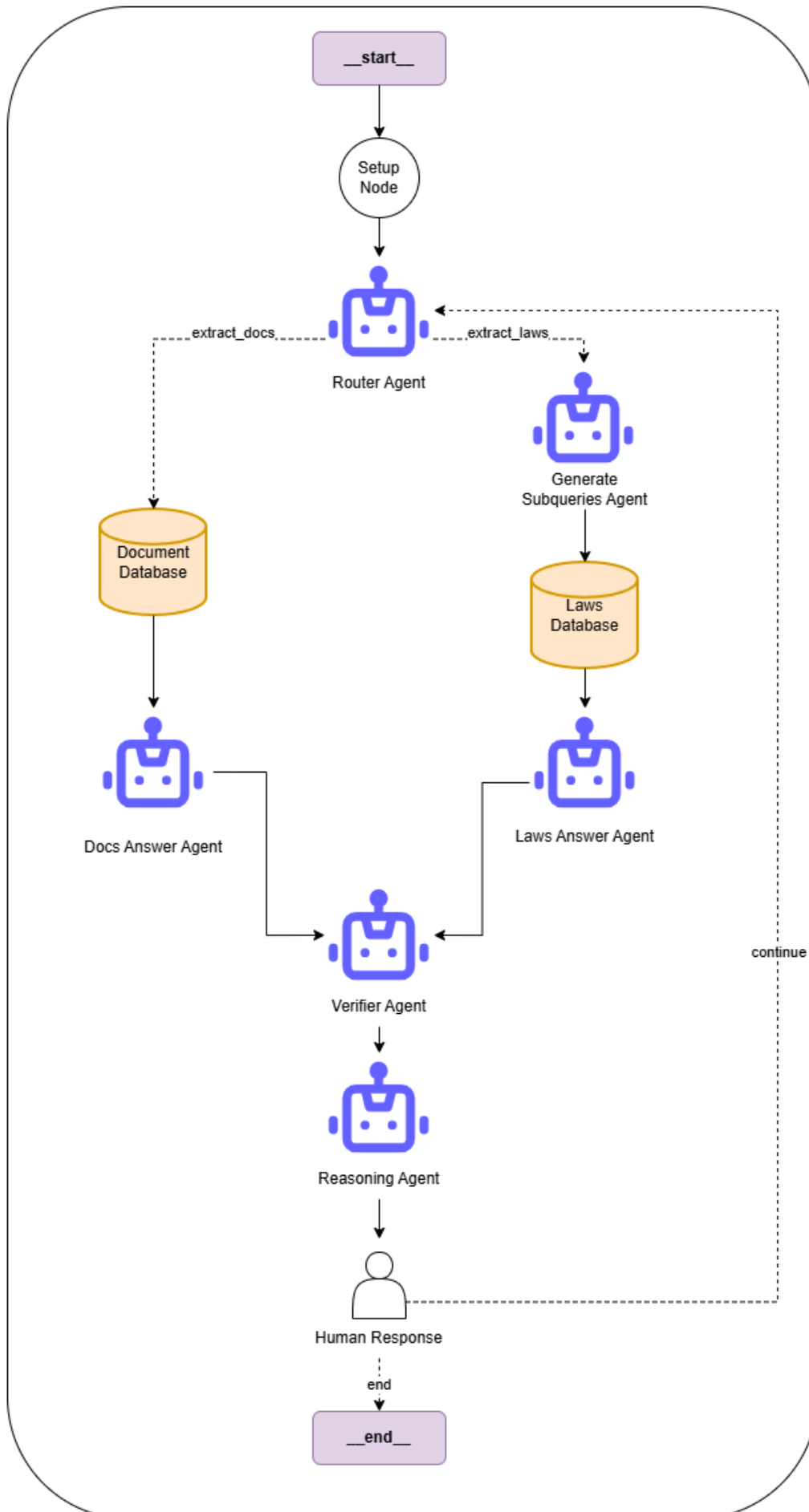


Figure 1: Overall architecture of our Multi Agent System

From the system, each component will be responsible for a specific task:

- **Setup Node:** Initializes the agent state and establishes the conversational memory. It is responsible for displaying the system’s welcome message and capturing the primary input query from the user to trigger the workflow.
- **Router Agent:** Acts as the central router for the system. It analyzes the semantic intent of the user’s query to determine the appropriate execution path, directing the flow either towards document processing (`extract_docs`) or laws information retrieval (`extract_laws`).
- **Generate Sub-queries Agent:** Enhances the retrieval process for the legal domain. It decomposes complex user inquiries into a list of precise sub-queries, optimizing the retrieval strategy to ensure comprehensive coverage of relevant legal concepts.
- **Laws Database:** The external knowledge base storing legal texts and regulations. It serves as the source for the system to fetch relevant legal context based on the generated sub-queries.
- **Documents Database:** Represents the contents of user-uploaded files. It is accessed by the system to extract specific content required to answer user questions regarding their personal documents.
- **Docs Answer Agent:** Specializes in handling queries related to uploaded files. It produces a response by processing the specific `docs_context` extracted from the provided documents.
- **Laws Answer Agent:** Specializes in answering queries based on reference laws. It constructs a response by synthesizing information from the retrieved `laws_context`, ensuring the answer is collected in the fetched legal data.
- **Verifier Agent:** Functions as a quality assurance agent. It evaluates the draft answers generated by the Docs Answer Agent or Laws Answer Agent to verify their accuracy and logic before presenting the final result to the user.
- **Reasoning Agent:** Provides explainability for the system’s answer-generation process. It generates a detailed ‘Chain of Thought’ explanation, describing the logical steps the system took to arrive at the final answer to the user.
- **Human Response:** Implements an interactive ‘*Human-in-the-loop*’ mechanism. It captures user feedback or follow-up questions, allowing the system to either continue the conversation via the Router Agent or terminate the session based on user intent.

2.2 Used Framework

The frameworks used in the system are illustrated bellow:

- **LangGraph:** This library serves as the core orchestration framework for the multi-agent system. It enables the creation of a cyclic stateful graph where agents act as nodes and transfer information via a shared `AgentState`. It can also handles the conditional routing logic and the ‘human-in-the-loop’ interaction flow.
- **LangChain:** Used as the primary interface for Large Language Model (LLM) interactions. It manages the communication with the Google Gemini API (`ChatGoogleGenerativeAI`), handles prompt construction (`SystemMessage`, `HumanMessage`), and integrates with `Pydantic` to enforce structured outputs for agents.

- **PyTorch**: Utilized to train and deploy the custom Neural Network model (PathClassifier) for the Router Agent. This implementation performs efficient, low-latency classification of user queries into “documents” or “extract_laws” categories without invoking an LLM for routing.
- **FAISS (Facebook AI Similarity Search)**: The high-performance vector database engine. It is used in two key areas:
 - **Static Indexing**: Storing and retrieving vectors for the static ‘**Laws Database**’.
 - **Dynamic Indexing**: Creating temporary, in-memory indices for uploaded PDF documents to enable real-time semantic search.
- **Sentence Transformers (Hugging Face)**: Responsible for generating dense vector embeddings from text input. These embeddings are essential for both the semantic retrieval process in FAISS and the input features for the PyTorch classification model.
- **Google Generative AI (Gemini 2.5 Flash)**: The underlying Large Language Model that powers the system’s generation capabilities. It is selected to handle tasks from generating sub-queries, producing query’s answer and verifying final answers.

3 Methodology

The project is implemented based on some core methodologies. This section will depict the methodologies used in the system.

3.1 Different types of Router

The Router serves as a fundamental component within the system, as its ability to determine the correct execution path directly influences the quality of the generated responses. Consequently, achieving high classification accuracy is essential. Functionally, the router operates as a binary classifier, categorizing user queries into either `extract_laws` or `extract_docs`. To optimize this process, various classification approaches were implemented and evaluated, allowing for the selection of the most effective model.

3.1.1 Key-word Matching Based

Keyword matching based router is the most basic routing approach, relying on string-matching mechanism. The system maintains a predefined dictionary of keywords associated with each intent. When receiving a user query, the router scans for the presence of these tokens to decide the route. While this method is computationally inexpensive and extremely fast, it lacks flexibility, failing to handle synonyms or understand the semantic context of complex queries.

3.1.2 Prompt-based Router

This method leverages the reasoning capabilities of Large Language Models (LLMs) like Gemini. A specific system prompt is designed to instruct the LLM to act as a classifier, analyzing the user’s intent and outputting a structured label (`extract_laws` or `extract_docs`). This approach benefits from the LLM’s deep natural language understanding, allowing it to handle ambiguous cases effectively. However, it introduces significant latency and higher operational costs due to the reliance on external API calls for every routing decision.

3.1.3 Classifier-based Router

This approach implements a specialized Supervised Machine Learning model—specifically a Feed-Forward Neural Network developed with PyTorch. The model is trained on a labeled dataset to map query embeddings (generated by a Sentence Transformer) to the correct class probabilities. This method represents the optimal balance for production systems: it achieves high classification accuracy (comparable to LLMs) but with drastically lower latency and resource consumption, as it runs locally without external dependencies.

3.1.4 Embedding Similarities Router

This technique utilizes Vector Space Modeling and semantic search. Both the user’s query and a set of “prototype” descriptions for each class are converted into high-dimensional vectors using an embedding model. The router calculates the Cosine Similarity between the query vector and the class prototypes. The query is directed to the class with the highest similarity score. This method captures semantic meaning better than keyword matching and is faster than LLM-based prompting, though it may struggle with subtle nuances compared to a trained classifier.

3.1.5 Evaluation Result

To assess the performance of the Router as a classification model, I employed the following standard evaluation metrics:

- **Precision:** Calculated independently for each class (`extract_laws` and `extract_docs`), this metric measures the accuracy of positive predictions - the proportion of true positives among all instances predicted as belonging to a specific class.
- **Recall:** Also computed per class, this metric evaluates the model’s ability to identify all relevant instances - the proportion of true positives detected out of all actual positives in that class.
- **F1-Score:** The harmonic mean of Precision and Recall. This metric provides a balanced assessment of the model’s performance for each class, especially useful if there is any class imbalance.
- **Accuracy:** The overall ratio of correctly classified queries to the total number of queries in the test set.

The comparative evaluation results for the different approaches are detailed below:

Method	Accuracy	Documents			Extract laws		
		Precision	Recall	F1	Precision	Recall	F1
Keyword matching based router	0.55	0.53	0.92	0.67	0.70	0.18	0.28
Classifier based router	0.97	0.96	0.99	0.97	0.99	0.96	0.97
Prompt based router	0.54	0.52	0.95	0.67	0.73	0.13	0.22
Embedding similarity router	0.98	1.00	0.97	0.98	0.97	1.00	0.98

3.2 RAG (Vector Database)

The system implements a dual-stream Retrieval-Augmented Generation (RAG) architecture designed to process both legal inquiries and document-based analysis. For the document workflow, user-uploaded files are processed with chunking and vector embedding before being indexed into the vector database. The legal module is built upon a comprehensive corpus of approximately **100,000 Vietnamese legal texts**, which are similarly embedded and stored.

A critical architectural distinction is the separation of these domains into two independent vector databases. To ensure high performance, specific optimization technologies are integrated to enhance retrieval efficiency and search productivity within these indices.

3.2.1 Reciprocal Ranking

To enhance the robustness of the legal information retrieval process, the system employs **Reciprocal Rank Fusion (RRF)**. RRF is a rank aggregation method used to combine the results of multiple search queries into a single, unified ranking. Unlike standard re-ranking methods, RRF relies solely on the rank position of documents, making it highly effective for merging retrieval results from diverse sub-queries. The underlying principle is that documents appearing consistently at the top of multiple search lists are more likely to be relevant than those appearing in only one. The score for a document d is calculated using the following formula:

$$\text{RRF}(d) = \sum_{q \in Q} \frac{1}{k + r(d, q)}$$

Where:

- Q : set of generated sub-queries.
- $r(d, q)$: rank position of document d in the retrieval list for query q (e.g., 1st, 2nd, 3rd).
- k : smoothing constant which mitigates the impact of high-ranking outliers.

In the context of my workflow, Reciprocal Ranking serves as the critical convergence point between query decomposition and final context generation:

- **Multi-Query Retrieval:** The `generate_subqueries_agent` first decomposes the user's complex legal question into 5 distinct sub-queries, each targeting a different legal aspect (e.g., conditions, procedures, penalties).
- **Parallel Execution:** The system performs a vector search for **each** of these 5 sub-queries against the FAISS Laws Database, producing 5 independent lists of candidate legal documents.
- **Fusion & Selection:** The `retrieve_laws_agent` applies the RRF algorithm to merge these lists. Documents that appear in the results for multiple sub-queries receive a higher cumulative score. The system then sorts the unique documents by this RRF score and selects the top 5 highest-ranked legal texts to serve as the context (`laws_context`) for the final answer generation.

This approach significantly ensures the final answer is comprehensive and grounded in multi-faceted legal evidence.

3.2.2 Index Implementation

3.2.2.1 Different Kinds of Indexing Strategies

To determine the optimal indexing structure for the legal retrieval system, I implemented and evaluated several vector indexing strategies. The primary objective was to balance retrieval accuracy (**Recall**) against computational efficiency (**Latency**) and memory usage, given a corpus of approximately 100,000 legal document embeddings. The details for the candidate indices are as follows:

- **Flat Index (FlatL2)**: The performance baseline, this index performs an *exhaustive brute-force search* by calculating L2 distances between the query and all vectors. While guaranteeing the highest possible accuracy, it is computationally expensive.
- **Inverted File Index (IVFFlat)**: This index accelerates search by partitioning the vector space into clusters. We configured it with `nlist=1024` (1024 clusters). During a search, the system only probes the most relevant cells, significantly reducing the search space compared to the Flat index.
- **Inverted File Index with Product Quantization (IVFPQ)**: This strategy compresses vectors to reduce memory footprint and speed up distance calculations. The query is searched with the most relevant cells (IVF) and then compare with PQ vectors in cells. We tested two configurations:
 - **IVFPQ_v1**: An initial configuration with `nlist=64`, `nbits=8` and `m=8` sub-quantizers.
 - **IVFPQ_v2**: An improved configuration with `nlist=256`, `nbits=8` and `m=16` sub-quantizers.
 - **IVFPQ_v3**: A highest-precision configuration with `nlist=1024`, `nbits=8` and `m=64` sub-quantizers, designed to improve recall over v2 while maintaining compression benefits.
- **Hierarchical Navigable Small World (HNSW)**: A graph-based indexing method with logarithmic search complexity. We implemented `IndexHNSWFlat` with `M=64` (neighbors per node), `efConstruction=256` (depth of search during graph construction) and `efConstruction=128` (depth of search during retrieval process). This configuration is optimized for high recall at very low latency.
- **Scalar Quantization**: This approach compresses standard 32-bit floating-point vectors into 8-bit integers (`QT_8bit`). We deployed both the standard `ScalarQuantizer` and the inverted version `IVFScalarQuantizer` (with `nlist=1024`) to evaluate the trade-off between precision loss from quantization and improvements in memory bandwidth.

3.2.2.2 Evaluation Results

To assess the performance of the aforementioned indices, the following Information Retrieval metrics were defined and measured:

- **Mean Reciprocal Ranking (MRR)**: Evaluates the ranking quality of the system. It is calculated as the average of the reciprocal ranks of the first relevant document retrieved ($\frac{1}{\text{rank}}$). An MRR closer to 1.0 indicates that the relevant legal document frequently appears at the very top of the list.
- **Precision**: The fraction of relevant documents among the retrieved documents. It measures the system’s ability to exclude irrelevant results from the output.

- **Recall:** The fraction of relevant documents that are successfully retrieved out of the total relevant documents available in the database. In the legal domain, **high recall is critical** to ensure no important regulations are overlooked.
- **Mean Average Precision (MAP):** The mean of the Average Precision (AP) scores for a set of queries. MAP provides a single-figure measure of quality across recall levels, rewarding systems that return relevant documents at higher ranks.
- **Average Time:** The average wall-clock time (latency) required to process a single query and retrieve results from the vector database, measured in seconds. This metric determines the real-time responsiveness of the system.

The comparative evaluation results for the different approaches are detailed below:

Indexing Method	k (Laws)	MRR	Precision	Recall	MAP	Average Time
IVFPQ_v1	10	0.15	0.03	0.25	0.14	0.013
IVFPQ_v2	10	0.39	0.07	0.59	0.36	0.011
	5	0.38	0.11	0.50	0.34	
IVFPQ_v3	10	0.54	0.09	0.78	0.52	0.014
	5	0.53	0.15	0.66	0.50	
Flat	10	0.69	0.10	0.88	0.67	0.036
	5	0.68	0.18	0.81	0.66	
HNSW	10	0.69	0.10	0.88	0.67	0.015
	5	0.68	0.18	0.81	0.66	
IVFFlat	10	0.68	0.10	0.86	0.66	0.015
	5	0.67	0.18	0.80	0.65	
ScalarQuantizer	10	0.69	0.10	0.88	0.67	0.029
	5	0.68	0.18	0.81	0.66	
IVFScalarQuantizer	10	0.68	0.10	0.86	0.66	0.014
	5	0.67	0.18	0.80	0.65	

From the table, we can witness a clear trade-off between retrieval accuracy and query latency across different indexing strategies. While the **Flat** index serves as the accuracy baseline (achieving the highest *Recall* of 0.88 and *MAP* of 0.67), it suffers from the highest latency (0.036s), making it less scalable. Conversely, quantization methods like **IVFPQ** significantly reduce retrieval time but at the cost of precision loss.

Notably, **HNSW** emerges as the optimal configuration for the legal retrieval system. It successfully maintains retrieval quality identical to the brute-force **Flat** index (*Recall*: 0.88, *MAP*: 0.67) while reducing inference time by approximately 58% (to 0.015s). This balance ensures that the system provides real-time responses without compromising the comprehensiveness required for legal documents search.

3.3 Multi Agent System Simulation with LangGraph

In order to simulate the multi-agent system, a graph is initialized with LangGraph. By using LangGraph, a mutual state is shared between all of the agents, which includes:

- **messages**: An annotated list of BaseMessage objects that stores the entire conversational history. It utilizes the add_messages reducer to ensure that new inputs from the user (HumanMessage) or outputs from agents (AIMessage) are appended to the sequence rather than overwriting it.
- **docs_context**: A string variable that holds the relevant text segments extracted from the user’s uploaded files. This context is populated by the Extract Docs Agent and serves as the grounding data for the Documents Answer Agent.
- **laws_context**: A string variable containing the synthetic legal texts retrieved from the vector database. It represents the results fetched by the Retrieve Laws Agent based on the generated sub-queries and reciprocal ranking.
- **uploaded_files**: A list of strings representing the file paths of documents uploaded by the user. This allows the system to track which files are available for processing during the session.
- **route**: A router string (specifically “documents” or “extract_laws”) determined by the Router Agent. This value acts as a route for the conditional edges in the graph, indicating which workflow branch to execute next.
- **generated_subqueries**: A list of strings containing the decomposed sub-questions produced by the Generate Sub-queries Agent. This state allows the retrieval agent to execute multiple targeted searches in parallel.

The state in LangGraph enables a decoupled architecture where agents function as independent units of logic. Instead of passing data directly from one agent to another, they simply modify this shared state. For instance, the Laws Agent does not need to know about the Retrieve Laws Agent; it simply looks for data in laws_context. This design ensures data persistence across the workflow and facilitates debugging by providing a snapshot of the system at any point in time.

4 Agent Details

The multi-agent system is composed of specialized agents, each designed to handle a specific stage of the . Below is the detailed specification of each agent, including their mechanism, input parameters, and output data.

4.1 Router Agent

The Router Agent acts as the system’s classifier, determining the user’s intent to direct the correct workflow. Unlike LLM-based routers, this agent utilizes a trained Feed-Forward Neural Network (PathClassifier) built with PyTorch. This model was built with **2 fully-connected layers**, **RELU activation function** and **dropout rate of 0.3** to prevent overfitting. The loss function I used is **CrossEntropyLoss** and the Optimizer is **AdamW**. When a query is pushed into model, it is embedded to a **1024-dimension vector** using “**intfloat/multilingual-e5-large**”. Then the vector is forwarded through two fully-connected layers to distribute the possibility for each class. Generally, queries will be

classified into two distinct categories: documents (for user-uploaded files) or `extract_laws` (for legal inquiries). The overall input and output of this model are illustrated below:

- **Input:** Messages containing the conversation history (specifically the last `HumanMessage`).
- **Output:** A string indicating the route ("`documents`" or "`extract_laws`").

4.2 Generate Sub-queries Agent

This agent is designed to enhance retrieval recall for laws database. It receives the user's query and strictly follows a system prompt to break it down without paraphrasing, ensuring comprehensive coverage of the legal issue. Initial user query will be decomposed into specific legal aspects. It employs ***Gemini-2.5-Flash*** LLM with structured output to generate five distinct sub-queries focusing on different aspects of laws. The overall input and output of this model are illustrated below:

- **Input:** `HumanMessage` (User's query).
- **Output:** A list of strings containing the original query plus 5 generated sub-queries.

4.3 Retrieve Laws Agent

This agent is responsible for fetching relevant legal context from the external database. It encodes the list of sub-queries into vectors, performs parallel searches in the FAISS laws index. It also implements a **Reciprocal Rank Fusion (RRF)** strategy to merge search results from multiple sub-queries to generate the final related laws to user's query. The overall input and output of this model are illustrated below:

- **Input:** `generated_subqueries` (from the `AgentState`).
- **Output:** `laws_context` (A string or list of identifiers representing the retrieved legal texts).

4.4 Laws Answer Agent

This agent is responsible for the actual answer generation for the legal workflow. It acts as a specialized legal consultant, synthesizing the final answer based strictly on the provided legal context from ***Retrieve Laws Agent***.. The overall input and output of this model are illustrated below:

- **Input:** `laws_context` and `Messages` (conversation history).
- **Output:** `AIMessage` containing the answer for user's query..

4.5 Extract Documents Agent

This agent manages the content for user-uploaded files. It handles the file ingestion, processing, and retrieval of information from PDF documents provided by the user during the session. It initially reads PDF files using `pypdf`, splits text into chunks using a sliding window approach (tokens), and builds a temporary, in-memory FAISS index (`IndexFlatL2`). Lastly, it embeds the user's query and performs a similarity search to find relevant text chunks. The overall input and output of this model are illustrated below:

- **Input:** `uploaded_files` list and the user's query.
- **Output:** `docs_context` (string containing most relevant text chunks extracted from the files).

4.6 Documents Answer Agent

Similar to the Laws Answer Agent, this agent is dedicated to answering questions based on the `docs_context`. It uses a system prompt that enforces strict adherence to the provided text and supports both Q&A and summarization tasks. The overall input and output of this model are illustrated bellow:

- **Input:** `docs_context` and conversation history.
- **Output:** `AIMessage` containing the answer derived from the uploaded documents.

4.7 Verifier Agent

The agent serves as a Quality Assurance (QA) layer of the system. It evaluates the answer generated by either the Laws or Documents Answer agent against the retrieved context to ensure accuracy and safety. Specifically, it performs four checks: Fact-check, Relevance-check, Clarity-check, and Policy-check. The original answer is decided whether to keep or regenerate. The overall input and output of this model are illustrated bellow:

- **Input:** `context` (laws or docs), the candidate `AIMessage`, and the original `HumanMessage` query.
- **Output:** A tuple containing `final_answer` (string) and an `explanation` (string) of the verification result.

4.8 Reasoning Agent

This agent is implemented to provide transparency and explainability within the system. It is responsible for generating a narrative describing the system’s internal decision-making process to provide the answer for user’s query. Functionally, it analyzes the entire execution trace, including the chosen route, the specific list of activated agents, and the context used to produce a detailed “*Chain of Thought*” explanation for the user. The overall input and output of this model are illustrated bellow:

- **Input:** The complete `AgentState` including route, context, and final answer.
- **Output:** A step-by-step explanation of how the system generated the result.

5 Evaluation and Experimental Testing

The system is evaluated on two key factors: **Performance** (Quality of generation/retrieval) and **Latency** (Processing time) for each agent.

5.1 Agent Performance

In terms of the performance metrics; **precision, recall, F1, mean reciprocal ranking and cosine similarities** are used. Apart from that, the evaluation utilizes the **DeepEval** framework, leveraging “LLM-as-a-judge” to score components. DeepEval is a simple-to-use, open-source LLM evaluation framework, for evaluating and testing large-language model systems. It incorporates the latest research to evaluate LLM outputs based on metrics such as G-Eval, task completion, answer relevancy, hallucination, etc. It is also designed for different types of LLMS: RAG, Chatbots, etc. It is this versatility that i decided to integrate this framework into the system’s evaluation.

5.1.1 Router Agent

-**Metrics Used:** Precision, Recall, F1-Score, and Accuracy.

- **Dataset:** A balanced test set comprising **2,000 queries** including 1,000 legal queries from the [Vietnamese legal](#) dataset and 1,000 general reading comprehension queries from [UIT-ViQUAD2.0](#) dataset.
- **Results:** As mentioned in the section 4.1.5; the result for the in-use classifier-based router can be seen bellow:

Class	Precision	Recall	F1-Score
Documents	0.96	0.99	0.97
Extract Laws	0.99	0.96	0.97
Overall Accuracy: 0.97			

The experimental results demonstrate that the router agent successfully balances high-precision retrieval with reliable, hallucination-free generation. It achieved a good classification accuracy (97%).

5.1.2 Generate Sub-queries Agent

- **Metrics Used:** For the Generate Sub-queries agent, two metrics are implemented:
 - **Cosine Similarity:** This metric evaluate if the generated queries relate to the original queries.
 - **Query Coverage** (GEval Framework): This metric evaluates whether the 5 generated sub-queries cover all necessary intents and aspects of the original complex query to ensure a comprehensive search.
- **Dataset:** 1000 legal queries sampled from [Vietnamese legal](#) dataset.
- **Results:** The result for Generate Sub-queries Agent can be seen bellow:

Metric	Score	Note
Relevance (Cosine Similarity)	0.93	Evaluated using all 1000 samples
GEval Score (Query Coverage)	0.90	Evaluated using 20 random samples

With a high **Cosine Similarity** score of **0.93** across 1,000 samples, the agent demonstrates strong semantic alignment between the original questions and the generated sub-queries. Furthermore, the **GEval Query Coverage** score of **0.90** out of **1** confirms that the decomposition strategy successfully captures the multifaceted nature of legal problems, ensuring that no critical aspect is overlooked during the retrieval process.

5.1.3 Retrieve Laws Agent

- **Metrics Used:** To evaluate the effectiveness of the legal information retrieval process, four standard metrics are employed:
 - **Mean Reciprocal Rank (MRR):** Measures the ability of the system to return the most relevant result at the top of the list. It is calculated as the average of the reciprocal ranks of the first relevant document retrieved across a set of queries Q :

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

Where:

- rank_i : rank position of the first relevant retrieved document for the i -th query.
- Q : set of evaluated queries.
- **Precision**: Measures the accuracy of the retrieved results, defined as the proportion of retrieved documents that are relevant to the user’s query.

$$\text{Precision} = \frac{|\{\text{relevant}\} \cap \{\text{retrieved}\}|}{|\{\text{retrieved}\}|}$$

Where:

- $\{\text{relevant}\}$: the set of actual relevant documents in the vector database
- $\{\text{retrieved}\}$: the set of retrieved document
- **Recall**: Assesses the exhaustiveness of the retrieval process, defined as the ratio of relevant documents successfully identified to the total number of relevant documents existing in the corpus. In the context of a legal RAG system, **maximizing recall is important**; it guarantees that no critical regulations are skipped, thereby ensuring the generated legal advice is grounded in a complete basis.

$$\text{Recall} = \frac{|\{\text{relevant}\} \cap \{\text{retrieved}\}|}{|\{\text{relevant}\}|}$$

- **Mean Average Precision (MAP)**: Provides a comprehensive measure of ranking quality, rewarding the system for placing relevant laws higher in the result list. It is calculated as the mean of the Average Precision (AP) scores across all queries. For a single query, the Average Precision (AP) is computed as:

$$\text{AP} = \frac{1}{|\text{Relevant}|} \sum_{k=1}^n (P(k) \times \text{rel}(k))$$

Where:

- $|\text{Relevant}|$: Total count of actual relevant documents for the query.
- $P(k)$: Precision at rank k .
- n : number of retrieved documents.
- $\text{rel}(k)$: Indicator function equal to 1 if the document at rank k is relevant, and 0 otherwise.

The Mean Average Precision (MAP) is computed as:

$$\text{MAP} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \text{AP}(q)$$

- **Dataset**: Due to the specificity of this agent, no suitable public benchmarks were available. Consequently, a custom evaluation dataset was constructed by **leveraging AI to generate 102 distinct queries paired with their corresponding relevant legal texts**, derived directly from our corpus of 100,000 Vietnamese legal documents index. This synthetic dataset underwent a manual verification process to ensure accu-

racy prior to the evaluation phase. The resulting dataset can be seen below: [Custom Dataset](#)

- **Results:** As mentioned in the section 4.2.2.2, the system utilizes the **HNSW (Hierarchical Navigable Small World)** index configuration for optimal speed and accuracy trade-off. The evaluation results (for 10 retrieved documents for each query ($k = 10$)) can be seen below:

Index	Precision	Recall	MRR	MAP
HNSW	0.88	0.1	0.69	0.67

The index HNSW achieves an acceptable **Recall** of **0.88**, ensuring that nearly 90% of relevant legal provisions are successfully retrieved. Furthermore, the **Mean Reciprocal Rank (MRR)** of **0.69** indicates that the most relevant law typically appears in the top 2 positions of the search results. Although the **Precision** is lower (**0.10**) due to recall-oriented strategy (10 documents are retrieved for each query), this trade-off is acceptable to guarantee comprehensive legal coverage.

5.1.4 Laws Answer Agent

- **Metrics Used:** To assess the quality of the this agent, we utilized the **DeepEval** framework to calculate two critical metrics:
 - **Faithfulness:** Measures the extent to which the generated answer is **derived truly from the retrieved context**, ensuring no hallucinations or fabrication of legal provisions. The context and generated answer are decomposed into claims. Then, it is calculated as:

$$\text{Faithfulness} = \frac{\text{Number of Claims Supported by Context}}{\text{Total Number of Claims in Generated Output}}$$

A score of 1.0 indicates that every claim made by the agent can be backed up by the retrieved legal documents.

- **Correctness:** Evaluates the factual accuracy of the generated answer by comparing it against a ground truth (expected output). It assesses whether the answer semantically aligns with the correct legal interpretation.

$$\text{Correctness} \in [0, 1]$$

Where the score is determined by an LLM judge based on the semantic overlap and factual consistency between the generated answer and the reference answer.

- **Dataset:** To evaluate this agent, we utilized a subset of the [vietnamese-legal-qa](#) dataset. Specifically, **10 random samples** (question-answer pairs and corresponding laws_contexts) were selected to undergo a rigorous manual and automated review process.
- **Results:** The evaluation results for the Laws Answer Agent are presented below:

Metric	Score
Faithfulness	1.0
Correctness	0.88

From the experimental results, it can be witnessed that the **Faithfulness** score of **1.0** is a critical achievement for a legal assistant, confirming that the strict prompt engineering (which explicitly forbids adding external knowledge) is working effectively. This ensures the system **does not hallucinate fake laws**. The **Correctness** score of **0.88** further validates that the agent is capable of reasoning correctly over the retrieved laws to provide accurate answer.

5.1.5 Extract Documents Agent

- **Metrics Used:** To key retrieval metrics in DeepEval is used to evaluate the Extract Documents Agent:

- **Contextual Precision:** Measures the proportion of the relevant information in the retrieved chunks that is actually useful for generating the answer. High precision indicates that the agent retrieves highly relevant content with minimal distraction.

$$\text{Contextual Precision} = \frac{\text{Relevant Chunks in Retrieval}}{\text{Total Retrieved Chunks}}$$

- **Contextual Recall:** This metric measures the completeness of the retrieval. It evaluates whether the retrieved context contains all the necessary information required to construct the expected answer.

$$\text{Contextual Recall} \in [0, 1]$$

A score of 1.0 implies that the retrieved context is sufficient to answer the user’s question completely without missing any key details.

- **Dataset:** As this agent also has high specificity, I also constructed a custom evaluation dataset comprising **10 collected PDF files** covering diverse topics to simulate real-world usage. For each file, i generated corresponding question-answer pairs using a **semi-automated approach** (AI generation followed by manual proof-checking) to ensure the ground truth was accurate and grounded in the documents. Each sample in the dataset includes:

- question: The user query.
- answer: The ground truth answer derived from the PDF.
- list of pdf-files: The list of 10 collected PDF files.

The resulting dataset can be seen below: [Custom Dataset](#)

- **Results:** The evaluation results for the Extract Documents Agent are presented below:

Metric	Score
Contextual Precision	0.95
Contextual Recall	0.87

From the results, i can conclude that the agent demonstrates exceptional **Contextual Precision (0.95)**, indicating that the chunking and embedding strategy (using Faiss FlatL2) is highly effective at filtering out irrelevant text segments. The **Contextual Recall (0.87)** confirms that in the vast majority of cases, the agent successfully retrieves the complete context needed to answer the user’s question.

5.1.6 Documents Answer Agent

- **Metrics Used:** Similar to the Laws Answer Agent, Documents Answer Agent’s assessment focuses on two primary dimensions derived from **DeepEval** framework: **Faithfulness** and **Correctness**
- **Dataset:** For this evaluation, we utilized the [UIT-ViQUAD2.0](#) dataset. We selected **10 random samples** to test the agent’s ability to comprehend and extract answers from general Vietnamese text passages. Each sample consists of a question, a docs_context (passage), and an expected_output (answer).
- **Results:** The evaluation results are summarized below:

Metric	Score
Faithfulness	1.0
Correctness	0.34

The **Faithfulness** score of **1.0** is a significant positive result, indicating that the system prompts successfully enforce the constraint to use only the provided context. Regarding the **Correctness** score of **0.34**, the lower value is primarily attributed to a structural mismatch between the dataset and the agent’s output. The **expected_output** in the test dataset typically consists of **short, direct extractions from the text** (e.g., just a name or a date). In contrast, our agent is designed to generate comprehensive, **full-sentence answers** to provide a natural conversational experience. Consequently, while the agent’s responses are factually accurate and more complete, this difference in verbosity and structure leads to a lower similarity score against the concise ground truth during evaluation.

5.2 Agent Processing Time

During system’s evaluation, each agent is also evaluated with average processing time for each query.

5.2.1 Experimental Environment

As latency’s evaluation depends strictly on hardware capacity, to better understanding about latency result, here is the information about my hardware’s specification while evaluating the system:

Component	Specification
CPU	Intel Core i5-14600K
GPU	NVIDIA RTX 4070 SUPER (12GB)
RAM	32GB DDR5
Storage	1TB NVMe SSD 5000MB/s

5.2.2 Experimental Result

The following table summarizes the average latency for each component in the system:

Agent Name	Average Processing Time (s/query)
Router Agent	0.04s
Extract Docs Agent	0.16s
Retrieve Laws Agent	0.28s
Documents Answer Agent	2.20s
Generate Sub-queries Agent	4.64s
Laws Answer Agent	6.99s

Note: Times for Extract Docs and Retrieve Laws and Router Agent exclude the initial query as it costs index and model initialization. The **cache mechanism is applied** to substantially reduce latency for next queries.

6 Link to Dataset and Source Code:

6.1 Dataset

The link to the dataset can be seen bellow:

- Vietnamese Legal Dataset: [Link](#)
- Vietnamese Documents QA Dataset: [Link](#)
- Vietnamese Legal QA Dataset: [Link](#)
- Custom Data for Extract Laws and Extract Docs Evaluation: [Link](#)

6.2 Source Code

The link to the source code can be found publicly on: [Github Link](#)

7 Discussion

7.1 Conclusion

This project has successfully designed and implemented a robust **Multi-Agent RAG System** for two distinct tasks: Vietnamese legal consultation and general document question-answering. By leveraging the **LangGraph** framework, the system effectively orchestrates specialized agents—including a Neural Network-based Router, targeted Retrieval agents, a safety-focused Verifier and an explainable Reasoning Agent to handle complex user queries with high precision.

7.2 Future Work

Upon the current prototype, several strategic directions are proposed to upgrade the system into a comprehensive solution. The immediate objective is to refine the current prototype into a **full-scale graduation thesis (Capstone Project)** by conducting more

extensive user testing, using more advanced techniques to handle complex scenarios. Simultaneously, the system's domain expertise will be ***expanded beyond laws*** to high-demand sectors such as Psychology, Education, etc, while enhancing the document pipeline to ***support multi-modal inputs*** like images. Finally, to address critical data privacy concerns, future developments will transition from external APIs to ***Local Large Language Models***. Hosting the model locally will ensure that sensitive user documents and legal queries remain within a secure infrastructure.